

AN INTRODUCTION FOR JAVA DEVELOPERS

Spring & Spring Boot

From core container concepts to building production-ready REST services

Core Spring · Annotations · Spring Boot · REST APIs

WHAT WE'LL COVER

Agenda

01

Why Spring

History, motivation, and the benefits that made it the Java standard

02

Core Spring

Inversion of Control, Dependency Injection, and bean configuration

03

Annotations

The annotation-driven style that replaced XML configuration

04

Spring Boot

Opinionated auto-configuration, starters, and embedded servers

05

Building REST Services

Controllers, request handling, and error handling — our main focus

WHY SPRING EXISTS

Java Development Before Spring

- Heavyweight EJB containers for simple business logic
- Sprawling XML descriptors just to wire objects together
- Classes reached out and constructed their own collaborators
- Nearly impossible to unit test without a running container

```
public class OrderService {  
    private OrderRepository repository;  
  
    public OrderService() {  
        // tightly coupled to one impl  
        this.repository =  
            new JdbcOrderRepository();  
    }  
}
```

WHY SPRING EXISTS

What Is Spring?

A lightweight framework for building Java applications out of plain old Java objects (POJOs) — no heavyweight container required.

Born from Rod Johnson's 2002 book *Expert One-on-One J2EE Design and Development*, which argued that simpler, testable objects beat EJB ceremony.

Inversion of Control

The container creates and wires your objects

Dependency Injection

Collaborators are supplied, not constructed

Aspect-Oriented Programming

Cross-cutting concerns stay out of business logic

WHY SPRING EXISTS

A Brief History

2004

Spring 1.0

XML-based IoC container ships, offering an alternative to EJB

2007

Spring 2.5

Annotation-driven configuration arrives: @Autowired, @Component

2009

Spring 3.0

Java-based configuration: @Configuration and @Bean replace XML

2013

Spring 4.0

Java 8 support and the @RestController annotation for MVC

2014

Spring Boot 1.0

Auto-configuration, starters, and embedded servers change everything

2017

Spring 5.0

Reactive programming with Spring WebFlux

2022

Spring 6 / Boot 3

Jakarta EE namespace, Java 17 baseline, native image support

Today

The default framework for enterprise Java

WHY SPRING EXISTS

Why It Became the Standard



Loose coupling

Objects depend on interfaces, not concrete implementations



Testability

Swap in mocks and test business logic without a container



Plain Java objects

No mandatory base classes or heavyweight interfaces



Modularity

Use only the modules you need — web, data, security, messaging



Huge ecosystem

Integrates cleanly with nearly every Java library and data store

Still evolving

20+ years of active development, from XML to reactive to native images

CORE SPRING

Inversion of Control

WITHOUT IOC

Your class controls its own flow: it decides which implementation to create, when, and how.

```
new OrderService(new JdbcRepo())
```

WITH IOC

The container controls the flow: it creates objects and hands them their dependencies.



```
container.getBean(OrderService.class)
```

CORE SPRING

Dependency Injection

Constructor injection is the recommended style: dependencies arrive as constructor parameters, so a bean can never exist half-wired.

```
@Service
public class OrderService {
    private final OrderRepository repository;
    private final PaymentClient paymentClient;

    @Autowired
    public OrderService(OrderRepository repository,
                        PaymentClient paymentClient) {
        // Spring resolves and injects both beans
        this.repository = repository;
        this.paymentClient = paymentClient;
    }
}
```

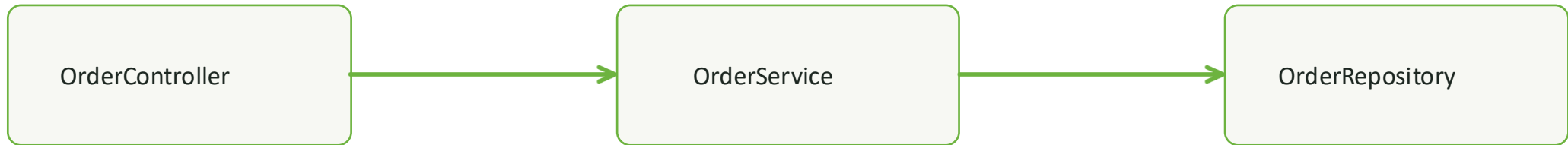
On a single constructor, **@Autowired** is optional as of Spring 4.3 — it's shown here for clarity

CORE SPRING

The ApplicationContext

Spring's IoC container. It reads your configuration, creates beans, injects their dependencies, and manages their entire lifecycle.

ApplicationContext



Beans are singletons by default — created once, cached, and shared everywhere they're needed

CORE SPRING

Three Ways to Configure Beans

Style	How it works	Era	Used today
XML	Beans declared in <bean> elements in an XML file	Spring 1.0+	Rarely
Java Config	@Configuration classes with @Bean factory methods	Spring 3.0+	For third-party beans
Annotations	Classes self-register with @Component and friends	Spring 2.5+	Default choice

Modern Spring and Spring Boot apps mix annotation-based components with a small amount of Java config for beans you don't own

CORE SPRING

Java-Based Configuration

Reach for `@Configuration` and `@Bean` when you need to wire a class you didn't write — a third-party client, for example.

```
@Configuration
public class PaymentConfig {

    @Bean
    public PaymentClient paymentClient(
        @Value("${payments.api-key}") String apiKey) {
        // Spring calls this method once and manages the result
        return new PaymentClient(apiKey);
    }
}
```

ANNOTATIONS

Why Annotations?

Annotations move configuration next to the code it describes, instead of into a separate XML file.

- Metadata lives beside the class or method it configures
- Less boilerplate, fewer files to keep in sync
- Convention over configuration — sensible defaults, override when needed
- The compiler and IDE can validate references

```
@Service
public class OrderService {
    // this class + its dependencies
    // are declared in one place
}
```

ANNOTATIONS

Stereotype Annotations

Each marks a class for auto-detection by @ComponentScan and gives it a role

Annotation	Marks	Layer
@Component	A generic, managed bean	Any
@Service	Business logic	Service
@Repository	Data access; adds automatic exception translation	Persistence
@Controller	Web MVC handler returning view names	Web
@RestController	@Controller + @ResponseBody — returns data, not views	Web

ANNOTATIONS

Wiring Beans Together

```
@Service
public class NotificationService {

    @Autowired
    @Qualifier("smtpSender")
    private MailSender mailSender;

    @Value("${notify.retry-count:3}")
    private int retryCount;
}
```

@Autowired

Injects a matching bean — on a field, setter, or constructor

@Qualifier

Picks one bean by name when several implementations exist

@Value

Injects a property, with an optional default value

@Primary

Marks the default bean when no @Qualifier is given

ANNOTATIONS

More Popular Annotations

@Configuration

Declares a class that defines beans via @Bean methods

@ComponentScan

Tells Spring which packages to scan for annotated classes

@Scope

Sets bean scope — singleton (default) or prototype

@PostConstruct

Runs a method right after dependency injection completes

@PreDestroy

Runs a method just before the bean is destroyed

@Profile

Activates a bean only for a given environment, like "dev"

ANNOTATIONS

A Peek at Aspect-Oriented Programming

AOP pulls cross-cutting concerns — logging, security, transactions — out of business logic and into one reusable place.

```
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.acme.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        // runs around every service method — no changes to the services themselves
        log.info(pjp.getSignature() + " took " + (System.currentTimeMillis() - start));
        return result;
    }
}
```

CHECKPOINT

Core Spring gives you loose coupling and testable code — with far less boilerplate than writing it all by hand.

IoC container

Dependency Injection

Annotation-driven beans

AOP

PART TWO

Spring Boot

Same Spring container underneath — with the setup work done for you

SPRING BOOT

Even Annotated Spring Was Still Work

- Pick compatible versions of a dozen libraries by hand
- Write @Configuration classes for common setups every project needs
- Install and configure an external servlet container
- Write your own health checks and metrics from scratch

The question Spring Boot asks

"What if most Spring apps agreed on sensible defaults — and you only configured what makes yours different?"

SPRING BOOT

What Is Spring Boot?

An opinionated layer on top of the Spring Framework — same container, same annotations, radically less setup.



Auto-Configuration

Beans configure themselves based on what's on the classpath



Starters

One dependency pulls in a curated, compatible library set



Embedded Server

Tomcat ships inside your jar — run it with `java -jar`



Actuator

Health checks, metrics, and ops endpoints out of the box

SPRING BOOT

Starter: One Dependency, Whole Stack

Starter	Brings in
spring-boot-starter-web	Spring MVC + embedded Tomcat
spring-boot-starter-data-jpa	Spring Data JPA + Hibernate
spring-boot-starter-security	Authentication and authorization
spring-boot-starter-validation	Bean Validation (jakarta.validation)
spring-boot-starter-test	JUnit, Mockito, Spring Test

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

SPRING BOOT

Starting an Application

```
@SpringBootApplication
public class CatalogApplication {

    public static void main(String[] args) {
        SpringApplication.run(CatalogApplication.class, args);
    }
}
```

@SpringBootApplication combines @Configuration, @EnableAutoConfiguration, and @ComponentScan into one annotation

Run it with **java -jar app.jar** — Tomcat starts embedded, no separate server install

SPRING BOOT

Externalized Configuration

One file per environment, read automatically — no code changes to move from dev to production.

```
application.yml

server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://db:5432/catalog
    username: ${DB_USER}

logging:
  level:
    com.acme: DEBUG
```

Read with @Value or @ConfigurationProperties

Bind whole sections of config to a typed Java object

Profile-specific overrides

application-prod.yml overrides values for the "prod" profile

Environment variables win

\${DB_USER} is resolved from the environment at startup

PART THREE • MAIN FOCUS

Building REST Services

From a bare controller to a production-shaped API

BUILDING REST SERVICES

Anatomy of a REST Controller

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;

    ProductController(ProductService service) {
        this.service = service;
    }
    // handler methods go here
}
```

@RestController

A @Controller whose methods return data (JSON), not view names

@RequestMapping

Sets the base path shared by every handler method below it

The constructor is injected automatically — no @Autowired needed on a single constructor

BUILDING REST SERVICES

Mapping HTTP Verbs

Annotation	Verb	Typical use
@GetMapping	GET	Read one resource or a collection
@PostMapping	POST	Create a new resource
@PutMapping	PUT	Replace a resource entirely
@PatchMapping	PATCH	Update part of a resource
@DeleteMapping	DELETE	Remove a resource

Each is shorthand for @RequestMapping(method = RequestMethod.GET), and so on

BUILDING REST SERVICES

Getting Data Out of a Request

```
@GetMapping("/{id}")
public Product getById(
    @PathVariable Long id) { ... }

@GetMapping
public List<Product> search(
    @RequestParam(required = false) String category) { ... }

@PostMapping
public Product create(
    @RequestBody @Valid ProductRequest request) { ... }
```

@PathVariable

Pulls a value out of the URL path, like the id in /products/42

@RequestParam

Reads a query string parameter, like ?category=books

@RequestBody

Deserializes the JSON request body into a Java object

@Valid

Triggers Bean Validation on the deserialized object

BUILDING REST SERVICES

Putting It Together: A Full Controller

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;

    ProductController(ProductService service) { this.service = service; }

    @GetMapping
    public List<Product> findAll() { return service.findAll(); }

    @GetMapping("/{id}")
    public Product findById(@PathVariable Long id) { return service.findById(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product create(@RequestBody @Valid ProductRequest request) {
        return service.create(request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

BUILDING REST SERVICES

The Service and Repository Layers

@RestController

HTTP in, HTTP out

@Service

Business rules, transactions

Repository

Talks to the database

```
public interface ProductRepository
    extends JpaRepository<Product, Long> {
    // Spring Data generates the implementation
    List<Product> findByCategory(String category);
}

@Service
public class ProductService {
    private final ProductRepository repository;

    ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public Product create(ProductRequest req) {
        return repository.save(req.toEntity());
    }
}
```

BUILDING REST SERVICES

Controlling the Response

Returning a plain object always sends HTTP 200. Use `ResponseEntity` when the status code or headers need to vary.

```
@GetMapping("/{id}")
public ResponseEntity<Product> findById(@PathVariable Long id) {
    return service.find(id)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}
```

Found → 200 OK with the product body · Not found → 404 with an empty body

BUILDING REST SERVICES

Handling Errors Gracefully

One class turns thrown exceptions into consistent JSON error responses — no try/catch in every controller method.

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ProductNotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleInvalid(MethodArgumentNotValidException ex) {
        return new ErrorResponse("INVALID_REQUEST", ex.getMessage());
    }
}
```

@RestControllerAdvice applies across every controller in the application

CHECKPOINT

A handful of annotations take you from an empty class to a working, well-behaved REST API.

@RestController

@GetMapping / @PostMapping

@RequestBody

@RestControllerAdvice

WRAP-UP

Key Takeaways

1

Spring solves coupling

IoC and Dependency Injection
keep your classes simple,
swappable, and testable

2

Annotations are the API

@Component, @Autowired,
@Configuration and friends are
how you'll talk to Spring daily

3

Boot removes the setup tax

Starters and auto-configuration
get you to running code in
minutes

4

REST is a few annotations away

@RestController and its
mapping annotations turn a
service into an API

THANK YOU

Questions?

Docs: spring.io/projects/spring-framework · spring.io/projects/spring-boot

Bootstrap a new project: start.spring.io